



Manual

1. 시스템 개요 및 지원 플랫폼 (Overview & Platforms)

지원 플랫폼 및 실행 파일 (Cross-Platform)

2. 사전 요구 환경 및 설치 가이드 (Prerequisites & Installation)

2.1 수동 주행 모드 (Manual Mode)

2.2 자율주행 모드 (Auto Mode) 필수 통신 환경 구축

① ROS-TCP-Endpoint 설치 및 실행

② `kimm_msgs` 커스텀 인터페이스 패키지 생성 및 빌드

3. 시뮬레이터 조작 및 주행 모드 (Driving Controls & Modes)

3.1 키보드 조작 단축키 (Keyboard Controls)

3.2 레이싱 휠 단축키 (Racing Wheel)

3.3 수동 주행 모드 (Manual Mode)

3.4 자율주행 모드 (Auto Mode - 'M' Key / 'A' Button)

3.5 실시간 텔레메트리 차트 패널 (Telemetry Chart Panel - 'TAB' Key)

4. 시나리오 편집 모드 (Scenario Edit Mode - 'E' Key)

4.1 시작 위치 재설정

4.2 기본 장애물 배치 조작법

4.3 보행자 시나리오 및 웨이포인트 경로 생성

5. 메인 ESC 메뉴 및 환경 설정 (ESC Menu & Configurations)

5.1 메인 ESC 메뉴 기능 개요 (ESC Menu Overview)

5.2 시뮬레이션 맵 전환 (Map Selection)

5.4 센서 설정 (Sensor Config - `sensor_config.json`)

① 기본 단일 센서 구성 (`default_sensor_config.json`)

② 다중 센서 확장 구성 (`custom_sensor_config.json`)

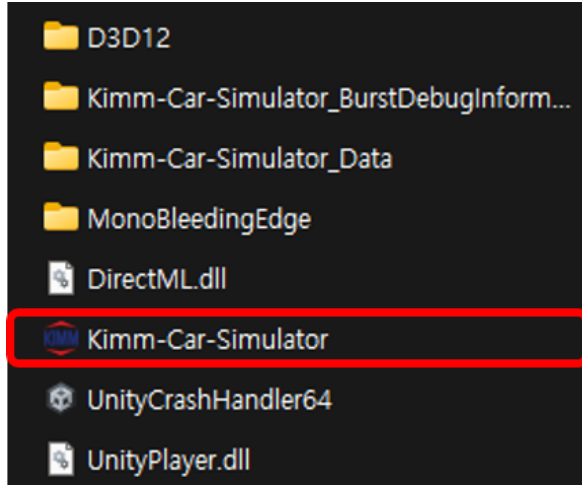
6. ROS2 인터페이스 명세 (ROS2 Interface)

MATLAB Simscape Multibody C++ FMU 동역학 및 ROS2 미들웨어 기반 차량 시뮬레이터 메뉴얼

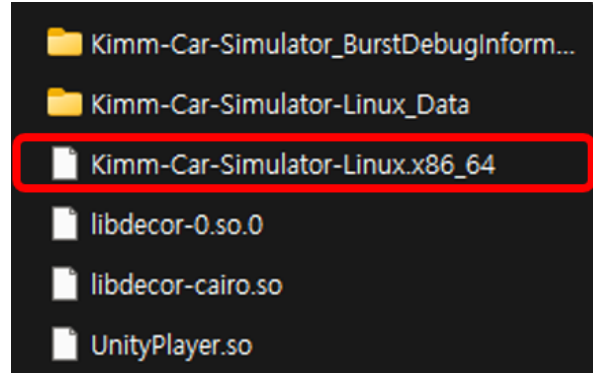
1. 시스템 개요 및 지원 플랫폼 (Overview & Platforms)

KIMM Car Simulator는 자율주행 및 차량 거동 연구를 위한 디지털 트윈 차량 시뮬레이터이다. MATLAB Simscape 차량 동역학 모델과 Unity 3D 그래픽 환경, 그리고 ROS2 통신 미들웨어를 연동하여 실차 수준의 주행 물리를 가상 환경에서 테스트할 수 있다.

지원 플랫폼 및 실행 파일 (Cross-Platform)



Windows build file



Linux Build file

플랫폼 (OS)	빌드 폴더명	실행 파일명	권장 사양
Windows	Kimm-Car-Simulator/	Kimm-Car-Simulator.exe	Windows 10/11 64-bit, RTX 3060 이상, RAM 16GB
Linux	Kimm-Car-Simulator-Linux/	Kimm-Car-Simulator-Linux.x86_64	Ubuntu 20.04/22.04 LTS, RTX 3060 이상, RAM 16GB

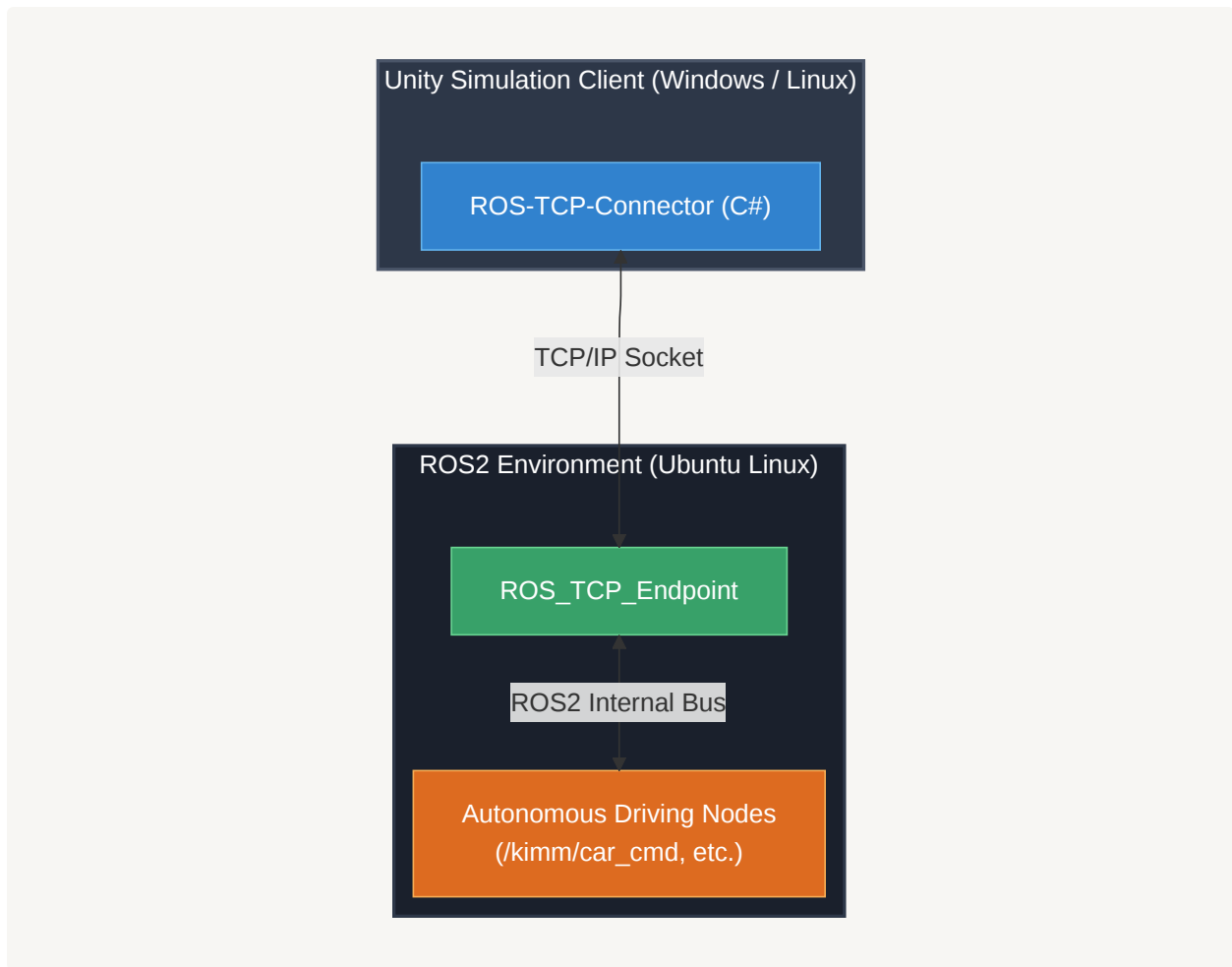
2. 사전 요구 환경 및 설치 가이드 (Prerequisites & Installation)

2.1 수동 주행 모드 (Manual Mode)

- **ROS2 연동 없이 단독 실행 가능:** 키보드, 마우스, 레이싱 휠, 게임패드를 연결하여 차량 동역학 시뮬레이션을 수행한다. 별도의 추가 패키지 설치 없이 빌드된 실행 파일(`Kimm-Car-Simulator.exe` 또는 `Kimm-Car-Simulator-Linux.x86_64`)을 즉시 실행하면 된다.

2.2 자율주행 모드 (Auto Mode) 필수 통신 환경 구축

Auto Mode에서 외부 ROS2 자율주행 노드(Autoware, Nav2, 자체 제어 알고리즘 등)와 양방향 데이터를 송수신하기 위해서는 **Unity-ROS2 TCP 통신 브릿지**(`ros_tcp_endpoint`)와 **커스텀 제어 메시지 패키지**(`kimm_msgs`)가 Ubuntu/Linux 환경에 반드시 설치 및 실행되어 있어야 한다.



① ROS-TCP-Endpoint 설치 및 실행

- **GitHub 공식 저장소:** [Unity-Technologies/ROS-TCP-Endpoint](https://github.com/Unity-Technologies/ROS-TCP-Endpoint)
- **도입 배경 및 필요성:**
 - Unity 3D 엔진(C#)과 ROS2(C++/Python DDS 미들웨어)는 서로 다른 런타임 환경에서 동작한다.
 - **ROS-TCP-Endpoint** 는 Unity 시뮬레이터와 ROS2 노드 사이에서 TCP/IP 네트워크 소켓을 통해 센서 데이터(PointCloud2, CompressedImage, Odometry) 및 제어 명령(CarControlCmd)을 실시간으로 고속 직렬화/역직렬화(Serialization/Deserialization)해 주는 공식 통신 게이트웨이 역할을 수행한다.
- **설치 및 빌드 절차 (Ubuntu/WSL2 터미널):**

```
# 1. ROS2 워크스페이스 src 디렉토리로 이동
mkdir -p ~/kimm_ws/src
cd ~/kimm_ws/src
```

```
# 2. ROS-TCP-Endpoint 저장소 클론 (ROS2 전용 main-ros2 브랜치)
git clone -b main-ros2 <https://github.com/Unity-Technologies/ROS-TCP-Endpoint.git>

# 3. 워크스페이스 빌드 및 환경변수 반영
cd ~/kimm_ws
colcon build --packages-select ros_tcp_endpoint
source install/setup.bash

# 4. 엔드포인트 브릿지 서버 실행 (기본 포트: 10000)
ros2 run ros_tcp_endpoint default_server_endpoint --ros-args -p ROS_IP:=127.0.0.1 -p ROS_TCP_PORT:=10000
```

② kimm_msgs 커스텀 인터페이스 패키지 생성 및 빌드

• 도입 배경 및 필요성:

- 일반적인 로봇용 속도 명령(`cmd_vel: linear, angular`)과 달리, 실제 승용차는 가속 페달(Throttle), 유압 브레이크(Brake), 조향각(Steering Angle), 변속기 기어(Gear: D/P/R)가 물리적으로 분리된 By-Wire 액추에이터 구조를 가진다.
- 실차 액추에이터 제어 명령을 1:1 직통 인가하기 위해 전용 메시지 인터페이스(`CarControlCmd.msg`)를 정의하여 사용한다.

• 메시지 정의 (`kimm_msgs/msg/CarControlCmd.msg`):

```
float32 accel      # 0.0 ~ 1.0 (가속 페달 개도율)
float32 steering   # -1.0(우회전) ~ 1.0(좌회전) (스티어링 정규화 조향각)
float32 brake      # 0.0 ~ 1.0 (유압 브레이크 제동 압력)
int32 gear         # 1: Drive(D), -1: Reverse(R), 0: Park(P)
```

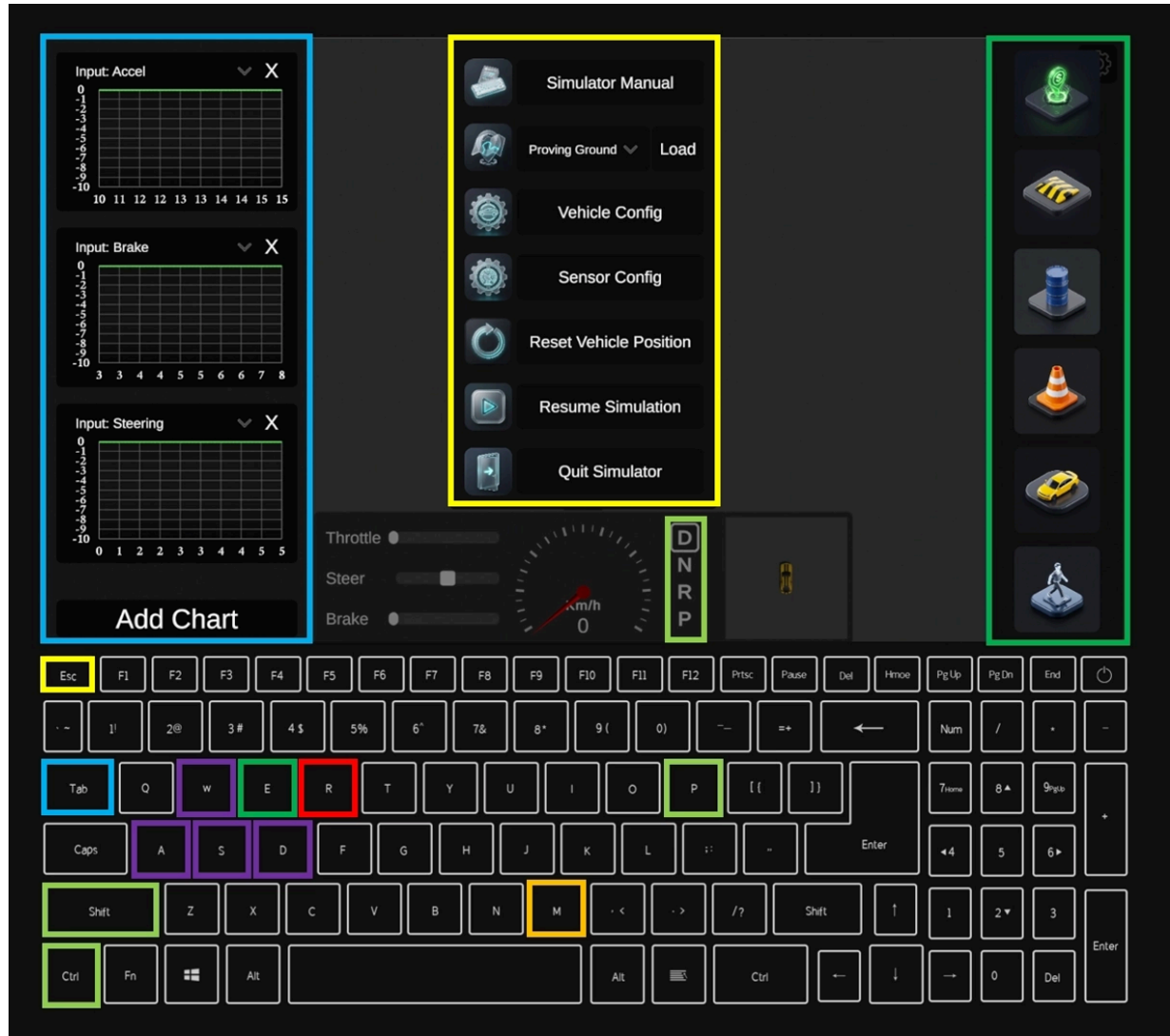
• 패키지 빌드 절차 (Ubuntu/WSL2 터미널):

```
# 1. 워크스페이스 빌드
cd ~/kimm_ws
colcon build --packages-select kimm_msgs
source install/setup.bash

# 2. 메시지 인터페이스 정상 생성 여부 확인
ros2 interface show kimm_msgs/msg/CarControlCmd
```

3. 시뮬레이터 조작 및 주행 모드 (Driving Controls & Modes)

3.1 키보드 조작 단축키 (Keyboard Controls)



키 (Key)	기능 (Function)	상세 설명
W / ↑	가속 (Accel)	전진 가속 페달 인가
S / ↓ / Space	브레이크 (Brake)	브레이크 제동 압력 인가
A / ←	좌회전 조향 (Steer Left)	스티어링 휠 좌측 조향
D / →	우회전 조향 (Steer Right)	스티어링 휠 우측 조향
Shift	전진 기어 (Drive - D단)	전진(D) 기어 변속 (gear: 1)
Ctrl	후진 기어 (Reverse - R단)	후진(R) 기어 변속 (gear: -1)
P	주차 브레이크 (Park - P단)	바퀴 회전 락 및 주차 제동 체결 (gear: 0)

키 (Key)	기능 (Function)	상세 설명
M	제어 모드 전환 (Manual ↔ Auto)	수동 주행(Manual)과 ROS2 자율주행(Auto) 1:1 토글
R	차량 위치 리셋 (Reset Vehicle)	지정된 스폰 위치 및 정지 상태로 차량 즉시 초기화
E	시나리오 편집 모드 (Edit Mode)	물리 연산 일시정지 및 장애물/보행자 편집 팔레트 오픈
TAB	텔레메트리 차트 패널 토글	실시간 동역학 수치(차속, 가속도, 슬립률) 그래프 On/Off
ESC	메인 메뉴 오픈 (ESC Menu)	차량 물리/센서 JSON 설정 교체, 맵 이동 패널 오픈
F5 ~ F8	카메라 시점 전환	1인칭/3인칭 뷰(F5), 탑뷰(F6), 좌측(F7), 우측(F8) 뷰 전환

3.2 레이싱 휠 단축키 (Racing Wheel)

레이싱 휠(Logitech G29, Power Shift Revolution 등)을 연결하면 별도의 매핑 설정 없이 즉시 주행 가능



휠 / 패드 하드웨어 (Input)	기능 (Function)	상세 설명
스티어링 휠 (Wheel / Left Stick)	아날로그 조향 (Steering)	조향각 비례 스티어링 제어
우측 가속 페달 (Gas Pedal / RT)	아날로그 가속 (Throttle)	페달 압력에 따른 가속도 비례 제어 (0 ~ 100%)

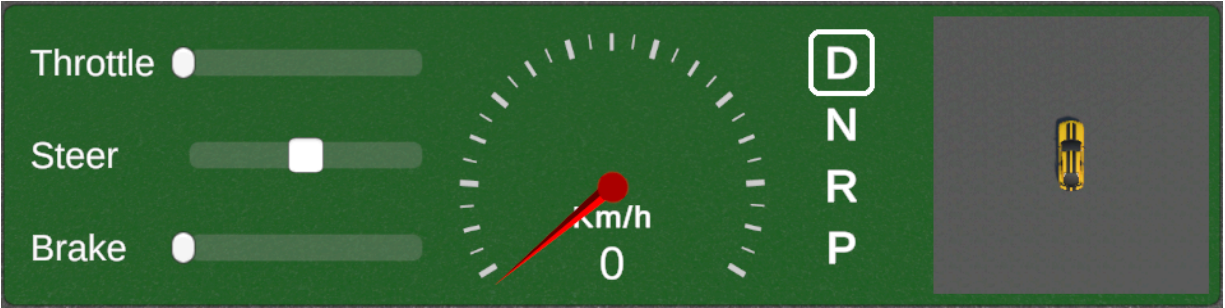
휠 / 패드 하드웨어 (Input)	기능 (Function)	상세 설명
좌측 제동 페달 (Brake Pedal / LT)	아날로그 제동 (Brake)	페달 압력에 따른 유압 제동력 비례 제어 (0 ~ 100%)
우측 패들 시프트 (Right Paddle / RB)	전진 변속 (Shift Drive)	전진(D) 기어 (gear: 1)
좌측 패들 시프트 (Left Paddle / LB)	후진/정지 변속 (Shift Reverse)	후진(R) ↔ 정지/파킹(P) 교대 순환
● A 버튼 (Button South)	모드 전환 (Manual ↔ Auto)	수동 주행과 자율주행 제어 모드 1:1 토글 ('M' 키 동일)
● B 버튼 (Button East)	차량 위치 리셋 (Reset Vehicle)	차량 스폰 지점 즉시 리셋 ('R' 키 동일)
● X 버튼 (Button West)	HUD 미니 카메라 시점 전환	텔레메트리 패널 내 미니 3D 뷰포트 시점 순환 변경
● Y 버튼 (Button North)	맵 순차 전환 (Next Map)	Proving Ground → K-City → Mcity → Zalazone 맵 순환 전환
⬆ 십자키 위 (D-Pad Up)	3인치 ↔ 1인치 뷰 전환	3인치 뷰 ↔ 운전석 1인치 뷰 토글 ('F5' 키 동일)
⬇ 십자키 아래 (D-Pad Down)	수직 탑뷰 (Top View)	수직 전체 관제 탑뷰 전환 ('F6' 키 동일)
⬅ 십자키 왼쪽 (D-Pad Left)	좌측 사이드 뷰 (Left View)	차체 좌측 수평 사이드 뷰 전환 ('F7' 키 동일)
➡ 십자키 오른쪽 (D-Pad Right)	우측 사이드 뷰 (Right View)	차체 우측 수평 사이드 뷰 전환 ('F8' 키 동일)

3.3 수동 주행 모드 (Manual Mode)

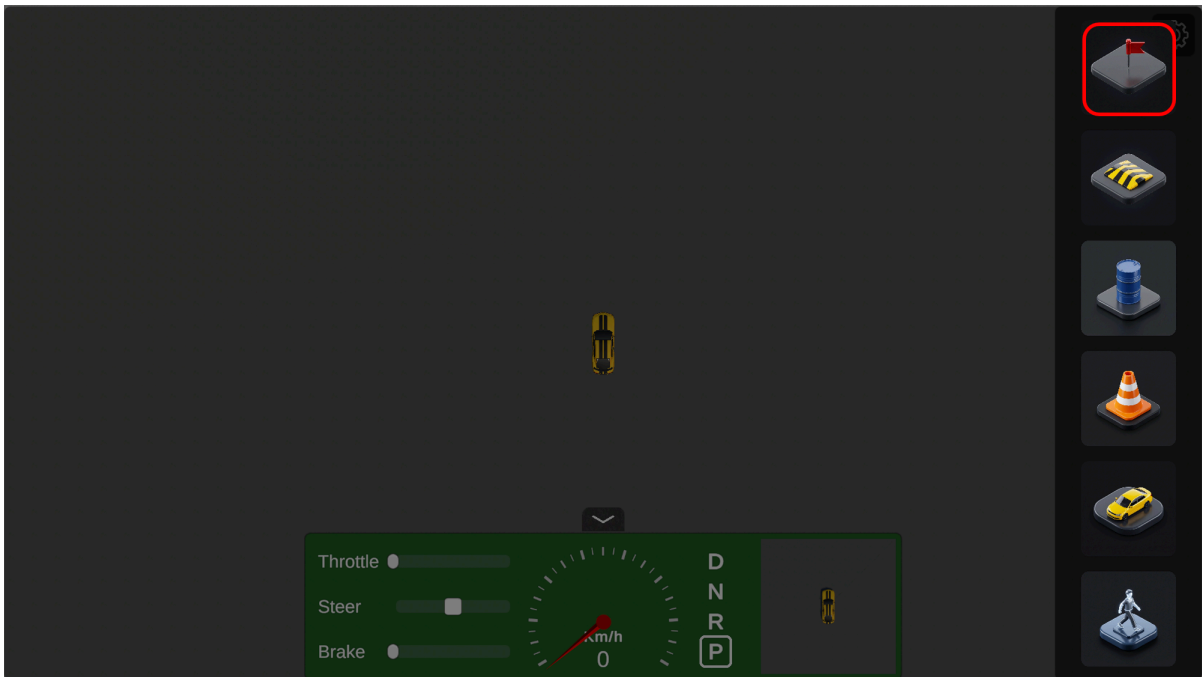


- HUD 테마: 검정색
- 운전자의 키보드, 레이스 휠 입력이 차량 FMU로 직접 입력된다.
- HUD 우측의 미니 뷰포트 클릭을 통해 시점을 순환 변경할 수 있다.

3.4 자율주행 모드 (Auto Mode - 'M' Key / 'A' Button)

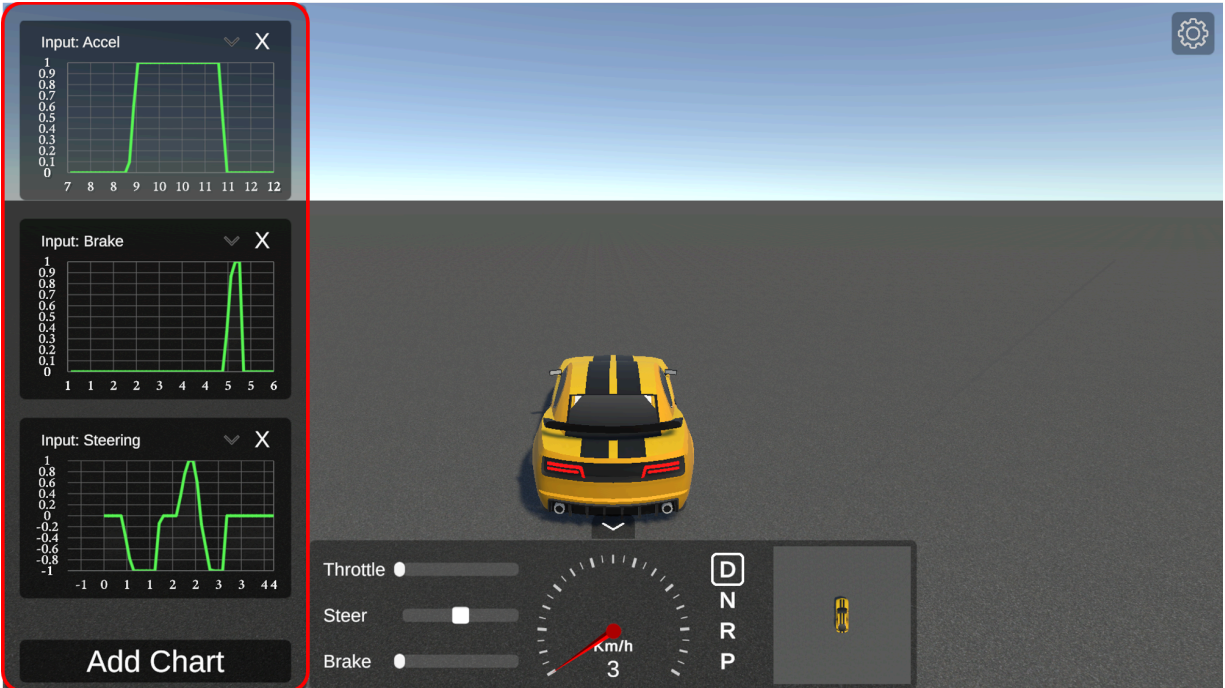


- **HUD 테마:** 초록색
- **수동 입력 차단:** 운전자 수동 입력이 비활성화되며, ROS2 `/kimm/car_cmd` 토픽으로 수신되는 제어 명령에 의해 차량이 구동된다.
- **목적지 지정 (Goal Pose Setting):**



1. **M** 키(또는 휠의 **A** 버튼)를 눌러 Auto Mode로 전환한다. (Manual Mode에서는 Spawn Point)
2. **E** 키를 눌러 편집 모드로 진입한 후 우측 최상단 핀 버튼(깃발 아이콘)을 선택한다.
3. 도로 노면 위 목적지를 **마우스 좌클릭**하여 배치한다. (마우스 휠로 도착 각도 회전)
4. 클릭 즉시 ROS2 `/kimm/goal_pose` 토픽으로 목적지가 발행되며, 내장 제어기 또는 외부 ROS2 노드에서 활용 가능하다.

3.5 실시간 텔레메트리 차트 패널 (Telemetry Chart Panel - 'TAB' Key)



주행 중 **TAB** 키를 누르면 화면 하단에 실시간 차량 FMU 파라미터 시각화 차트 패널이 토글된다.

- 동적 멀티 차트 추가:

- 패널 우측 상단의 **+** (Add Chart) 버튼을 클릭하여 원하는 개수만큼 차트 창을 자유롭게 추가 생성할 수 있다.
- 불필요한 차트는 차트 창 우측 상단의 **x** 버튼을 눌러 개별 삭제할 수 있다.

- 표시 파라미터 드롭다운 선택:

- 각 차트 상단의 드롭다운 메뉴를 통해 보고자 하는 동역학 수치를 1:1로 선택 및 실시간 전환할 수 있다:

4. 시나리오 편집 모드 (Scenario Edit Mode - 'E' Key)



주행 중 **E** 키를 누르면 물리 연산이 정지되며, 우측 팔레트 UI를 통해 도로 환경을 편집할 수 있다.

- [우측 에디트 팔레트 구성]
- └─ 1. Pin Button (Manual: 스폰 포인트 / Auto: Goal Pose)
 - └─ 2. Bump (과속방지턱)
 - └─ 3. Drum (드럼통 장애물)
 - └─ 4. Cone (라바콘)
 - └─ 5. Vehicle (주정차 더미 차량)
 - └─ 6. Pedestrian (웨이포인트 보행자)

4.1 시작 위치 재설정

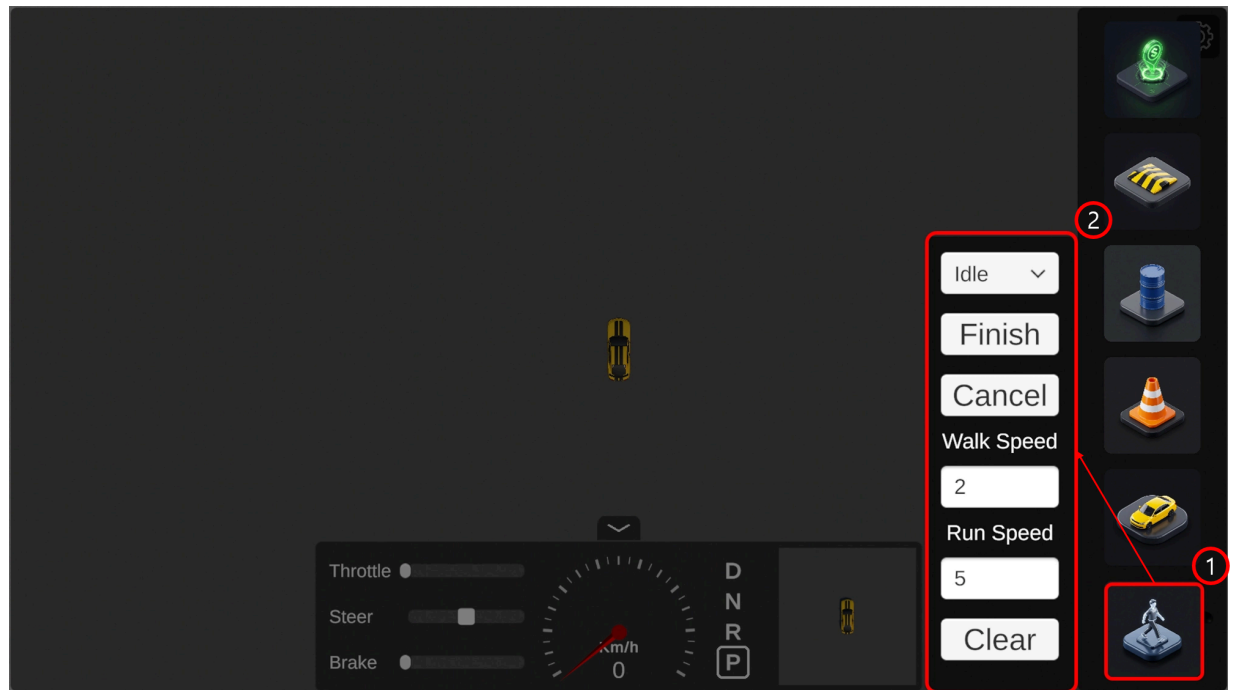
- **아이콘 클릭:** 편집모드 UI 최상단의 **Pin Button** (초록색 핀 모양 버튼) 클릭
- **카메라 시점 이동:** W/A/S/D 를 통해 원하는 시작 위치로 이동
- **마우스 이동:** 도로 3D 표면을 스냅 추종
- **마우스 휠 스크롤:** 5° 단위 3D 회전.
- **마우스 좌클릭:** 해당 위치로 차량 이동

4.2 기본 장애물 배치 조작법

- **아이콘 클릭:** 배치할 오브젝트 고스트(Ghost) 활성화.
- **카메라 시점 이동:** **W/A/S/D** 를 통한 시점 자유 이동
- **마우스 이동:** 도로 3D 표면을 스냅 추종.

- **마우스 휠 스크롤:** 5° 단위 3D 회전.
- **마우스 좌클릭:** 해당 위치에 오브젝트 생성.
- **마우스 우클릭:** 배치된 오브젝트 위에 마우스 커서를 올린 후 우클릭 시 삭제.

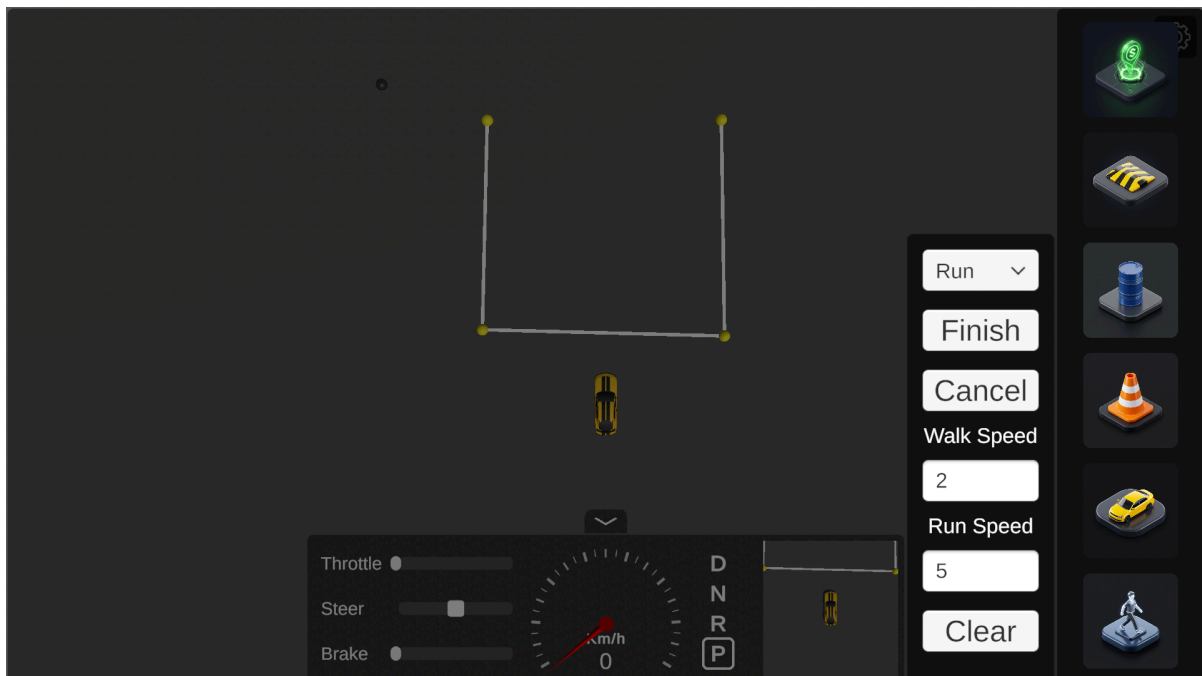
4.3 보행자 시나리오 및 웨이포인트 경로 생성



1. 팔레트에서 **Pedestrian** 아이콘을 클릭한다.
2. 보행자의 상태(Idle-대기/Walk-걷기/Run-뛰기)를 선택한다. (혼합 가능)



3. 보행자가 이동할 경로를 따라 도로 바닥을 **연속 좌클릭**하여 웨이포인트를 생성한다.



4. UI에서 보행자의 **보행 속도(Walk Speed)** 및 **달리기 속도(Run Speed)**를 입력한다.

5. **Finish** 버튼을 누르면 보행자가 생성되며, 주행 모드(**E** 키 복귀) 진입 시 경로를 따라 이동한다.

6. Clear 버튼을 통해 모든 보행자를 삭제할 수 있다.

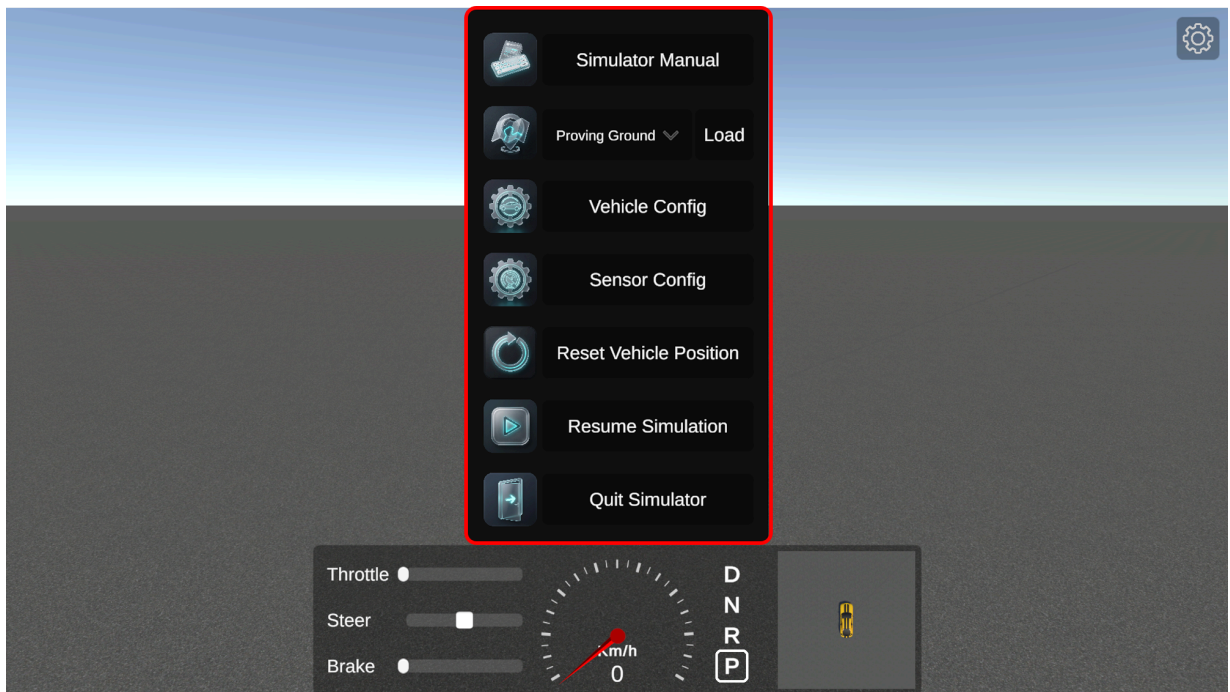
5. 메인 ESC 메뉴 및 환경 설정 (ESC Menu & Configurations)

주행 중 **ESC** 키를 누르면 시뮬레이션 물리 연산이 일시정지(`Time.timeScale = 0`)되며 메인 ESC 설정 패널이 화면에 활성화된다. 이 메뉴를 통해 소스코드 재빌드 없이 **시뮬레이션 맵 이동, 차량 물리 파라미터 변경, 센서 배치 핫 스왑(Hot-Swap)** 을 런타임에 즉시 수행할 수 있다.

[ESC 메인 메뉴 UI 구조 및 주요 기능]

- └─ 1. Map Selection (맵 드롭다운) : Proving Ground ↔ K-City ↔ Mcity 씬 즉시 이동
- └─ 2. Select Vehicle Config 버튼 : 파일 탐색기로 45개 차량 물리 JSON 실시간 교체/로드
- └─ 3. Select Sensor Config 버튼 : 파일 탐색기로 단일/다중 센서 JSON 실시간 교체/로드
- └─ 4. Reset Vehicle 버튼 : 차량을 현재 맵의 최초 스폰 지점으로 즉시 리셋 ('R' 키 동일)
- └─ 5. Controls Manual 버튼 ('F1') : 전체 단축키 및 레이싱 휠 조작 도움말 팝업 토글
- └─ ▶ 6. Resume 버튼 ('ESC') : 시뮬레이션을 재개하고 주행 모드로 복귀

5.1 메인 ESC 메뉴 기능 개요 (ESC Menu Overview)



- **조작 도움말 (F1 키 / Simulator Manual)**: 키보드 및 레이싱 휠 조작 매핑 가이드를 화면 중앙에 오버레이 팝업으로 띄운다.
- **차량 위치 리셋 (Reset Vehicle Position)**: 사고 발생 또는 경로 이탈 시 차량을 현재 맵의 안전한 시작 스폰 위치로 즉시 초기화한다.

- 시뮬레이션 재개 (**Resume** / **ESC** 키): 변경된 설정을 유지한 채 물리 연산을 재개하고 주행 모드로 복귀한다.

5.2 시뮬레이션 맵 전환 (Map Selection)

ESC 메뉴 상단의 **Map** 드롭다운 메뉴를 통해 원하는 테스트 트랙으로 즉시 이동할 수 있다.

맵 명칭 (Track Name)	특징 및 환경 구성
Proving Ground (PG)	평탄한 다목적 차량 종합 성능 시험장 및 직선/선회로
K-City (자율주행 실험도시)	한국교통안전공단 실도로 기반 교차로, 건물, 터널 트랙
Mcity (도심 자율주행 트랙)	도심형 인터체인지 및 복합 도로 환경 트랙
ZalaZone(잘라존 시험장)	유럽형 스마트시티 및 고속 핸들링/선회 시험 트랙



Tip

USB 레이싱 휠을 사용 중인 경우, 핸들의 **Y 버튼(Button North)** 을 누르면 ESC 메뉴를 열지 않고도 다음 맵으로 1초 만에 즉시 순환 전환된다.

• 빌드 실행 파일 기준 경로:

- Windows: `Kimm-Car-Simulator_Data/StreamingAssets/VehicleConfigs/`
- Linux: `Kimm-Car-Simulator-Linux_Data/StreamingAssets/VehicleConfigs/`
- 런타임 핫 스왑: **Select Vehicle Config** 버튼을 클릭하여 파일 탐색기에서 원하는 차량 JSON을 선택하면 즉시 실시간으로 파라미터가 갱신 및 재적용된다.
- 중요 주의사항: 아래 45개 파라미터 중 단 하나라도 누락되거나 변수명이 불일치할 경우 FMU 모델 초기화 오류가 발생하므로, 반드시 45개 변수가 모두 정의되어 있어야 한다.

```
{
  "Metadata": {
    "VehicleName": "KIMM Standard Sedan",
    "Description": "45개 표준 파라미터 설정 파일",
    "Version": "1.0"
  },
  "Parameters": {
    "Veh_AeroArea": 2.594,
    "Veh_AeroCd": 0.2888,
    "Veh_AeroCl": 0.149,
    "Veh_AeroRho": 1.205,

    "Veh_BodyInertia[1,1]": 600.0,
```

```
"Veh_BodyInertia[1,2]": 3000.0,  
"Veh_BodyInertia[1,3]": 3200.0,  
  
"Veh_BodyMass": 1600.0,  
"Veh_BodyRefZ0": 0.6147,  
"Veh_BodytoWheelCenter": 0.2647,  
"Veh_FrontAxleX": 1.5,  
"Veh_RearAxleX": -1.5,  
"Veh_SteerRatio": 16.0,  
  
"Veh_SuspF_BumpC": 3000.0,  
"Veh_SuspF_BumpK": 200000.0,  
"Veh_SuspF_BumpLimit": -0.084,  
"Veh_SuspF_BumpWidth": 0.003,  
"Veh_SuspF_C": 1750.0,  
"Veh_SuspF_EqPos": 0.1126,  
"Veh_SuspF_K": 35000.0,  
"Veh_SuspF_ReboundC": 3000.0,  
"Veh_SuspF_ReboundK": 200000.0,  
"Veh_SuspF_ReboundLimit": 0.056,  
"Veh_SuspF_ReboundWidth": 0.003,  
"Veh_SuspF_UnsprungInertia[1,1]": 1.0,  
"Veh_SuspF_UnsprungInertia[1,2]": 1.0,  
"Veh_SuspF_UnsprungInertia[1,3]": 1.0,  
"Veh_SuspF_UnsprungMass": 48.0,  
  
"Veh_SuspR_BumpC": 4000.0,  
"Veh_SuspR_BumpK": 300000.0,  
"Veh_SuspR_BumpLimit": -0.095,  
"Veh_SuspR_BumpWidth": 0.003,  
"Veh_SuspR_C": 2200.0,  
"Veh_SuspR_EqPos": 0.0985,  
"Veh_SuspR_K": 40000.0,  
"Veh_SuspR_ReboundC": 4000.0,  
"Veh_SuspR_ReboundK": 300000.0,  
"Veh_SuspR_ReboundLimit": 0.04,  
"Veh_SuspR_ReboundWidth": 0.003,  
"Veh_SuspR_UnsprungInertia[1,1]": 1.0,  
"Veh_SuspR_UnsprungInertia[1,2]": 1.0,  
"Veh_SuspR_UnsprungInertia[1,3]": 1.0,  
"Veh_SuspR_UnsprungMass": 45.0,
```

```

    "Veh_TrackF": 1.6,
    "Veh_TrackR": 1.6
  }
}

```

5.4 센서 설정 (Sensor Config - `sensor_config.json`)

- 빌드 실행 파일 기준 경로:

- Windows: `Kimm-Car-Simulator_Data/StreamingAssets/SensorConfigs/`
- Linux: `Kimm-Car-Simulator-Linux_Data/StreamingAssets/SensorConfigs/`

- 런타임 핫 스왑: `Select Sensor Config` 버튼을 클릭하여 파일 탐색기에서 원하는 센서 JSON을 선택하면 즉시 실시간으로 센서 인스턴스가 재배포 및 생성된다.

- 좌표계 표준: `FLU Standard` (Forward: +X, Left: +Y, Up: +Z)

① 기본 단일 센서 구성 (`default_sensor_config.json`)

단일 LiDAR 및 단일 카메라로 구성된 기본 설정이다.

```

{
  "SensorConfigName": "KIMM Default Sensor Suite",
  "ROS2Connection": {
    "RosIP": "127.0.0.1",
    "RosPort": 10000
  },
  "GNSS": {
    "FrameId": "gnss_frame",
    "TopicName": "/kimm/gnss/fix",
    "PublishFrequency": 10.0,
    "InitialLatitude": 0.0,
    "InitialLongitude": 0.0,
    "InitialAltitude": 0.0,
    "LocalPosition": { "x": 0.0, "y": 0.0, "z": 0.0 },
    "LocalRotation": { "roll": 0.0, "pitch": 0.0, "yaw": 0.0 }
  },
  "IMU": {
    "FrameId": "imu_frame",
    "TopicName": "/kimm/imu/data",
    "PublishFrequency": 100.0,

```

```

    "LocalPosition": { "x": 0.0, "y": 0.0, "z": 0.0 },
    "LocalRotation": { "roll": 0.0, "pitch": 0.0, "yaw": 0.0 }
  },
  "LiDAR": {
    "FrameId": "lidar_frame",
    "TopicName": "/kimm/lidar/points",
    "PublishFrequency": 20.0,
    "PointsNumPerScan": 100000,
    "MinRange": 0.1,
    "MaxRange": 70.0,
    "GaussianNoiseSigma": 0.02,
    "MaxIntensity": 255.0,
    "LocalPosition": { "x": 0.0, "y": 0.0, "z": 0.6 },
    "LocalRotation": { "roll": 0.0, "pitch": 0.0, "yaw": 0.0 }
  },
  "Camera": {
    "FrameId": "camera_frame",
    "TopicName": "/kimm/camera/color/compressed",
    "CameraInfoTopic": "/kimm/camera/camera_info",
    "PublishFrequency": 30.0,
    "ResolutionWidth": 640,
    "ResolutionHeight": 480,
    "FieldOfView": 60.0,
    "LocalPosition": { "x": 0.3, "y": 0.0, "z": 0.7 },
    "LocalRotation": { "roll": 0.0, "pitch": 0.0, "yaw": 0.0 }
  }
}

```

② 다중 센서 확장 구성 (`custom_sensor_config.json`)

LiDAR나 카메라를 2개 이상 배치하려면, `LiDAR` 및 `Camera` 키 아래에 **대괄호 배열**(`[ {1번}, {2번} ]`) 형태로 센서 객체를 나열한다. 키 이름 뒤에 복수형 `s` 를 붙일 필요 없이 `LiDAR`, `Camera` 단일 명칭 그대로 사용하면 되며, 시뮬레이터 엔진이 런타임에 정의된 개수만큼 센서 인스턴스를 자동으로 생성하고 독립된 토픽 및 TF를 연결한다.

```

{
  "SensorConfigName": "KIMM Dual-Camera & Dual-LiDAR Suite",
  "ROS2Connection": {
    "RosIP": "127.0.0.1",
    "RosPort": 10000
  }
}

```

```

},
"LiDAR": [
  {
    "FrameId": "front_lidar_frame",
    "TopicName": "/kimm/lidar_front/points",
    "PublishFrequency": 10.0,
    "PointsNumPerScan": 50000,
    "LocalPosition": { "x": 0.3, "y": 0.0, "z": 0.6 },
    "LocalRotation": { "roll": 0.0, "pitch": -15.0, "yaw": 0.0 }
  },
  {
    "FrameId": "rear_lidar_frame",
    "TopicName": "/kimm/lidar_rear/points",
    "PublishFrequency": 10.0,
    "PointsNumPerScan": 50000,
    "LocalPosition": { "x": -0.3, "y": 0.0, "z": 0.6 },
    "LocalRotation": { "roll": 0.0, "pitch": -15.0, "yaw": 180.0 }
  }
],
"Camera": [
  {
    "FrameId": "front_camera_frame",
    "TopicName": "/kimm/camera/color/compressed",
    "CameraInfoTopic": "/kimm/camera/camera_info",
    "PublishFrequency": 30.0,
    "ResolutionWidth": 640,
    "ResolutionHeight": 480,
    "FieldOfView": 90.0,
    "LocalPosition": { "x": 1.2, "y": 0.0, "z": 1.5 },
    "LocalRotation": { "roll": 0.0, "pitch": -15.0, "yaw": 0.0 }
  },
  {
    "FrameId": "rear_camera_frame",
    "TopicName": "/kimm/camera_rear/color/compressed",
    "CameraInfoTopic": "/kimm/camera_rear/camera_info",
    "PublishFrequency": 30.0,
    "ResolutionWidth": 640,
    "ResolutionHeight": 480,
    "FieldOfView": 110.0,
    "LocalPosition": { "x": -1.2, "y": 0.0, "z": 1.4 },
    "LocalRotation": { "roll": 0.0, "pitch": -10.0, "yaw": 180.0 }
  }
]

```

```

    }
  ]
}
```

6. ROS2 인터페이스 명세 (ROS2 Interface)

아래 토픽 목록은 기본 설정(`default_sensor_config.json`)을 기준으로 한다.

토픽명 (Topic Name)	메시지 타입 (Type)	주파수 (Hz)	설명
<code>/kimm/vehicle_status</code>	<code>nav_msgs/msg/Odometry</code>	50.0	차속, 오도메트리 위치, 쿼터니언 회전
<code>/kimm/lidar/points</code>	<code>sensor_msgs/msg/PointCloud2</code>	20.0	3D 라이다 포인트 클라우드 (100,000 pts/scan)
<code>/kimm/camera/color/compressed</code>	<code>sensor_msgs/msg/CompressedImage</code>	30.0	전방 RGB 카메라 JPEG 이미지 스트림 (640x480)
<code>/kimm/gnss/fix</code>	<code>sensor_msgs/msg/NavSatFix</code>	10.0	WGS84 위도, 경도, 타원체 고도 데이터
<code>/kimm/imu/data</code>	<code>sensor_msgs/msg/Imu</code>	100.0	3축 각속도 및 3축 가속도 데이터
<code>/kimm/goal_pose</code>	<code>geometry_msgs/msg/PoseStamped</code>	이벤트	Auto Mode 도로 갯발 클릭 시 발행
<code>/kimm/car_cmd</code>	<code>kimm_msgs/msg/CarControlCmd</code>	100.0	외부 자율주행 노드로부터의 차량 제어 명령 (수신)

© 2026 Mobility System Control Lab(MSCL). All Rights Reserved.